

Language Design in Practice

Comparing Imperative Python, Functional Python, and
Asynchronous TypeScript

COS 497 Independent Study

Final Technical Monograph

Vivek Reddy Bhimavarapu

08/11/2026

Abstract

Programming paradigms do not eliminate complexity, instead they determine where it appears and which tools programmers use to control it. This paper compares imperative Python, strictly functional Python, and asynchronous TypeScript through memory management, type safety, and concurrent execution. The comparison is grounded in two implementation artifacts: a Python lexer and recursive-descent expression parser, and a portfolio of Python functions written without loops, comprehensions, mutation, or variable reassignment. The parser shows how formal grammar rules determine precedence, associativity, error handling, and abstract syntax tree structure. The functional portfolio shows how immutability, recursion, and higher-order functions improve local reasoning while sometimes increasing allocation, stack use, and syntactic density. TypeScript adds compile-time checking to JavaScript but erases types before runtime, while its promise-based asynchronous model provides concurrency without automatically providing parallel CPU execution. The analysis finds that imperative Python is generally the clearest choice for direct stateful algorithms, functional techniques are strongest where predictable transformations and reduced shared state matter, and asynchronous TypeScript is especially effective for typed coordination of I/O bound work. A practical design should therefore select and combine paradigms according to the dominant risks of the problem rather than treating any paradigm as universally superior.

1 Introduction

A programming language does more than provide syntax for expressing an algorithm. Its type system, memory model, and control-flow mechanisms influence which errors are caught, how state is represented, and how easily a program can coordinate multiple activities. Programming paradigms add another layer to these decisions. An imperative program describes a sequence of state-changing operations; a functional program emphasizes expressions, pure functions, and new values; and an asynchronous program divides work around operations that may complete later. Each approach can solve many of the same problems, but each makes different costs visible to the programmer.

This project compares imperative Python, strictly functional Python, and asynchronous TypeScript through three criteria: memory management, type safety, and concurrent execution. Python provides a useful controlled comparison because the same dynamically typed language can support both ordinary imperative code and a deliberately restricted functional style. TypeScript provides a contrasting case because it adds compile-time type analysis to JavaScript while retaining JavaScript’s runtime behavior. TypeScript’s documentation explains that types are erased after checking and do not alter the behavior of emitted JavaScript [1]. TypeScript can therefore detect many inconsistent uses of known values before execution, but it cannot remove the need to validate untrusted values at runtime.

The comparison is grounded in two software artifacts. The first is a lexer and recursive-descent parser for mathematical expressions. It demonstrates how a formal grammar becomes executable control flow and how source text becomes a structured abstract syntax tree (AST). The second is a portfolio of Python exercises written without `for` or `while` loops, comprehensions, mutation, or variable reassignment. It demonstrates the benefits and costs of imposing functional constraints on a language that does not require them.

Together, the artifacts support the central argument of this paper: paradigms relocate complexity rather than remove it. Imperative Python favors direct control, strict functional Python favors predictable value transformations, and asynchronous TypeScript favors explicit coordination of nonblocking operations. Their value therefore depends on the kind of correctness, performance, and concurrency a program requires.

2 Language Design and Evaluation Criteria

Imperative, functional, and asynchronous programming are not mutually exclusive categories. Python and TypeScript are multi-paradigm languages, and a real application may use all three approaches in different modules. The categories in this study instead identify the dominant rule used to structure a solution. Imperative Python organizes behavior through statements, branches, loops, calls, and frequently mutable variables or collections. Functional Python treats computation primarily as the evaluation and composition of functions. Under the strict rules used by this project, functions return new values instead of changing existing

ones, and iteration is expressed through recursion or higher-order functions such as `map`, `filter`, and `reduce`. Asynchronous TypeScript organizes operations around promises and `async/await`, allowing a function to suspend its own progress while other work continues.

Memory management is the first criterion because each style creates and retains objects differently. Python manages objects in a private heap [2]. CPython combines reference counting with a cyclic garbage collector [3]. A programmer is therefore freed from explicit allocation and deallocation in ordinary Python code, but algorithm design still affects allocation. An imperative routine may update a list in place, while a functional routine may repeatedly create slices and new lists to preserve immutability. Automatic management prevents many manual-memory errors; it does not make copying, retention, or object lifetime irrelevant.

Type safety is the second criterion. Python determines types at runtime. Its annotations document intent and support external type checkers, but the Python runtime does not enforce function or variable annotations [4]. Functional Python inherits this same type system; purity does not by itself make a dynamically typed value statically safe. TypeScript checks assignments, parameters, return values, and object shapes before or during compilation, then erases those types when producing JavaScript [1]. The resulting distinction is important: TypeScript can reject an inconsistent use of a known value before execution, but external JSON, user input, and other runtime data still require validation.

Concurrency and parallelism form the third criterion, and the terms must be separated. Concurrency means multiple activities can make progress during overlapping periods. Parallelism means multiple instructions actually execute at the same time. Python offers threads, processes, and `asyncio`; the standard documentation describes `asyncio` as a library for concurrent, particularly I/O-bound, code using `async` and `await` [5]. In the standard CPython build, the Global Interpreter Lock (GIL) permits only one thread at a time to execute Python bytecode [6]. Processes, free-threaded builds, or specialized native code are alternatives when CPU-bound parallel work is required. JavaScript normally uses an event loop in which queued jobs run one at a time to completion. Its asynchronous operations are concurrent, but CPU-intensive work still blocks the executing agent unless workers or another parallel mechanism are used [7]. These distinctions prevent the misleading assumption that

`async` is a synonym for faster or parallel execution.

3 Parsing Theory and the Expression Parser

The parser artifact illustrates a foundational language-design problem: a computer must convert a flat sequence of characters into a structure that reflects meaning. This process is divided into lexical and syntactic analysis. Lexical analysis groups characters into tokens. For the expression `2 + 3 * x`, the lexer produces a number token, a plus token, another number token, a multiplication token, an identifier token, and an end-of-input marker. The lexer in `Week1-2/math_parser.py` also records each token's original position. That choice improves error reporting because an invalid character or malformed expression can be tied to a source location rather than reported only as a generic failure.

Syntactic analysis determines whether the token sequence follows the grammar and, if so, builds its structure. The project uses the following grammar:

```
expression -> term (("+" | "-") term)*
term -> unary (("*" | "/" ) unary)*
unary -> ("+" | "-") unary | power
power -> primary ("^" unary)?
primary -> NUMBER | IDENTIFIER | "(" expression ")"
```

The grammar encodes precedence through its layers. An **expression** can combine **term** values with addition or subtraction, while a **term** must finish parsing multiplication or division before control returns to the expression rule. Consequently, `2 + 3 * x` produces an addition node whose right child is a multiplication node, preserving the conventional interpretation `2 + (3 * x)`. Parentheses invoke the expression rule recursively from **primary**, allowing the programmer to override default grouping.

The implementation is a recursive-descent parser because a small group of mutually supporting methods descends through the grammar. `_expression`, `_term`, `_unary`, `_power`, and `_primary` correspond directly to nonterminals. This creates traceability between specification and code: a reader can compare a grammar rule with the function that implements it. The parser maintains a current-token index, advances when `_match` recognizes an expected

token type, and raises `ParserError` when the next token cannot continue a valid derivation. After `_expression` returns, `parse` requires the end-of-input token. That final check prevents a valid prefix from hiding invalid trailing text.

Associativity requires an additional design decision. Addition, subtraction, multiplication, and division are implemented with loops that repeatedly combine the existing left expression with the next operand. This makes them left-associative: $8 - 3 - 2$ becomes $(8 - 3) - 2$. Exponentiation instead uses a conditional followed by a recursive call through `_unary`, so $2 \wedge 3 \wedge 2$ becomes $2 \wedge (3 \wedge 2)$. The relationship between `_unary` and `_power` also gives exponentiation higher precedence than unary minus. As a result, -2^2 is represented as the negation of 2^2 , not as $(-2)^2$. These cases show that a grammar is not merely a list of accepted symbols; its organization determines the structure and meaning assigned to an expression.

The output is an abstract syntax tree rather than a concrete reproduction of every source token. Frozen data classes represent `Number`, `Identifier`, `UnaryOperation`, and `BinaryOperation` nodes. Punctuation such as parentheses does not need its own node because its purpose has already been captured by tree structure. Figure 1 presents the essential AST for $2 + 3 * x$.

```
BinaryOperation(+)  
|-- Number(2)  
'-- BinaryOperation(*)  
    |-- Number(3)  
    '-- Identifier(x)
```

Figure 1. Abstract syntax tree for $2 + 3 * x$.

This abstraction makes later stages possible. An evaluator, optimizer, formatter, or code generator can operate on node types without repeating lexical analysis. The artifact stops after producing JSON, but the AST is already an interface between parsing and any later interpretation.

The parser also demonstrates the connection between design and verification. The repository contains eleven parser and lexer tests covering tokenization, invalid characters,

precedence, associativity, unary operators, parentheses, identifiers, decimal numbers, JSON conversion, incomplete expressions, and empty input. On August 13, 2026, all eleven tests passed under Python 3.12.13 in 0.002 seconds. The evidence should not be overstated: these tests sample important cases rather than prove the grammar correct for every possible string. Nevertheless, the combination of a compact grammar, directly corresponding parser functions, immutable AST nodes, and focused tests makes the implementation auditable. It shows how formal rules become a practical program and establishes a baseline for comparing broader paradigm choices.

4 Imperative Python

Imperative Python describes computation as an ordered sequence of operations. The parser provides a direct example. `Lexer.tokenize` keeps a mutable character index and token list, advances through a `while` loop, and appends each discovered token. `Parser._expression` and `Parser._term` similarly maintain a current token index while incrementally replacing a local `expression` variable with a larger tree. This organization closely mirrors the operational question a reader asks: what token is being examined, what state changes next, and when does the routine stop?

That directness is a major strength. A mutable accumulator or loop often expresses a stateful algorithm with fewer intermediate allocations than a rule that requires a new collection for every update. Error handling is also explicit. The lexer raises `LexerError` at the exact unexpected character, and the parser raises `ParserError` when grammar expectations fail. Python exceptions move control to a caller that can report the error without threading error values through every return type.

Imperative Python does not require manual memory release. The runtime owns allocation and garbage collection, while local variables hold references to objects. However, mutation creates aliasing: two variables may refer to the same list or dictionary, so changing it through one name changes what the other observes. Aliasing is efficient when intentional, but it expands the amount of program state a reader must consider. The parser limits this risk by using frozen data classes for AST nodes even though its control flow is imperative. This

mixed design demonstrates that paradigms can be combined: a mutable cursor is convenient during parsing, while immutable output nodes protect the completed result.

Python’s dynamic typing offers similar flexibility. `parse_expression` accepts a declared `str` and returns the `Expression` union, and the annotations make this contract legible to tools and readers. At runtime, however, annotations are not enforcement [4]. A caller can still pass a value of an unexpected type and receive a runtime failure. In a small program, rapid execution and concise syntax may justify that trade-off. In a large program, a static checker can use the same annotations to catch inconsistencies earlier, but this remains a development workflow choice rather than an inherent runtime guarantee.

Python also offers several concurrency models rather than one universal mechanism. Threads work well for some blocking I/O and can share memory, but shared mutation requires synchronization and the standard CPython GIL limits simultaneous Python-bytecode execution [6]. Processes enable CPU-bound parallel execution at the cost of separate memory spaces and interprocess communication. `asyncio` schedules coroutines cooperatively and is well suited to high-level network and I/O work [5]. The imperative style remains useful in all three models, but shared state becomes more dangerous as the number of independently progressing activities grows.

The imperative approach is therefore strongest when the algorithm is naturally stateful, execution order is central, or in-place work materially improves clarity or efficiency. Its weakness is not mutation itself but uncontrolled mutation. A disciplined design can use direct state changes within a narrow scope and expose immutable or well-defined results, as the parser does.

5 Strict Functional Python

The functional portfolio applies a stronger rule than ordinary idiomatic Python. Its implementation contains no `for` or `while` statements, no comprehensions, no deliberate input mutation, and no variable reassignment. Instead, it uses recursion, `map`, `filter`, `functools.reduce`, and function composition. The restriction is valuable as an experiment because it makes the consequences of the paradigm visible; it should not be confused with a claim that every

production Python function ought to follow the same rule.

Pure functions are the central benefit. A pure function's result depends on its inputs and it does not change externally visible state. `squares`, for example, maps a transformation over its input and returns a new list. `recursive_reverse` returns a new reversed list without changing the sequence it receives. `quicksort` selects a pivot, filters the remaining values into new groups, recursively sorts those groups, and concatenates new lists. A caller can retain the original input with confidence, and a test can evaluate the function without constructing or cleaning up hidden state.

Higher-order functions express common traversal patterns without explicit loops. `even_numbers` passes a predicate to `filter`; `product` passes multiplication to `reduce`; and `group_by` reduces records into a new dictionary. `compose` returns a function that applies supplied functions from right to left, while `pipe` applies them from left to right. Functions are therefore both behavior and data: they can be accepted, returned, stored, and combined.

This style improves local reasoning. Because a function does not mutate a shared collection, a reader does not need to search for every alias that might observe an intermediate state. The benefit becomes especially relevant under concurrency. Independent transformations of immutable inputs require less coordination than workers updating the same dictionary. Functional design does not automatically make concurrent code correct, but it removes a major class of shared-state interactions.

The same rules impose measurable costs. Python does not optimize tail recursion, and its recursion depth is finite. The recursive `fibonacci` function uses an accumulator-based helper, but every call still consumes a Python stack frame. Recursive sorting and sequence functions repeatedly slice collections. `recursive_sum(values[1:])` constructs a new slice for each step, and `recursive_reverse` additionally concatenates a new list at each return. These choices preserve the assignment's restrictions but may use more time and memory than an imperative loop over the original sequence. The code is appropriate for demonstrating recursive decomposition, not for processing an arbitrarily large list.

Persistent updates to dictionaries create another trade-off. `frequency` combines the accumulator with a one-entry dictionary using the union operator on each reduction step. This avoids mutating the accumulator but creates a new dictionary repeatedly. An imperative

implementation using `counts[item] += 1` would usually allocate less and may be easier for a Python programmer to recognize. Functional style makes the state transition explicit as a value, while imperative style performs it more economically in place.

Readability also depends on the audience and operation. `squares` and `even_numbers` are concise because `map` and `filter` name familiar transformations. The nested conditional expression in recursive merge sort or the tuple construction inside `average_by` is denser than an ordinary loop. Eliminating a statement does not necessarily eliminate conceptual complexity; sometimes it compresses that complexity into an expression.

The portfolio's tests reflect both results and constraints. Nine test methods cover recursive algorithms, error cases, sorting and searching, higher-order transformations, collection operations, composition, text processing, matrices, and record aggregation. On August 13, 2026, all nine tests passed under Python 3.12.13 in 0.007 seconds. Tests also verify that sorting leaves its input unchanged. The final test parses `functional_exercises.py` with Python's own `ast` module and rejects `for`, `async for`, `while`, and all comprehension node types. This is an effective structural check because a future edit cannot silently introduce the prohibited syntax while still passing output tests.

The AST check does not prove semantic purity. Code could mutate an object through a method call without containing a loop, and a called library function could have side effects. Conversely, the check prohibits comprehensions even though a comprehension may construct a new value without mutating its input. The test therefore proves compliance with the assignment's syntactic constraints, not with every philosophical definition of functional programming. That limitation is acceptable because the enforced rule is explicit and independently testable.

Strict functional Python is most useful here as a lens. It reveals the advantages of deterministic transformations, makes input preservation easy to verify, and encourages reusable higher-order operations. It also reveals why practical Python programs usually combine paradigms: mutation within a well-contained function can be clearer and more efficient, while immutable interfaces can preserve most of the reasoning benefits.

6 Asynchronous TypeScript

TypeScript contributes two distinct features to the comparison: static analysis and JavaScript's asynchronous execution model. Its structural type system checks whether values have the required properties and operations. An asynchronous function can declare a result such as `Promise<User[]>`, allowing a checker to verify that callers `await` or otherwise handle the promised collection. This contract is stronger before execution than an unenforced Python annotation. It can catch misspelled properties, inconsistent return paths, and many accidental uses of a promise as though it were its resolved value.

The guarantee has a boundary. TypeScript erases type annotations and emits JavaScript [1]. A network response claimed to be a `User[]` does not become trustworthy because the programmer wrote a type assertion. Runtime validation remains necessary at external boundaries. TypeScript's contribution is therefore early consistency checking within the modeled program, not proof that every runtime value matches its declaration.

JavaScript promises represent eventual completion. Calling an `async` function returns a promise, and `await` suspends the surrounding `async` function until the awaited promise settles [8]. It does not block the entire thread. Other queued work can proceed while a network request, timer, or other asynchronous platform operation is pending. `async/await` makes dependency order resemble synchronous code and allows ordinary `try/catch` around rejected operations.

Concurrency must still be expressed correctly. Consider two independent requests. Awaiting the first before starting the second makes them sequential. Starting both and passing their promises to `Promise.all` permits their waiting periods to overlap. MDN notes that promise combinators support concurrent composition, while JavaScript still executes one job at a time on a given agent [9]. The performance benefit comes from avoiding idle waiting, not from executing two CPU-heavy JavaScript functions simultaneously.

The event loop also provides run-to-completion behavior: each queued job finishes before the next job begins [7]. This reduces some low-level interleavings within a job, but asynchronous boundaries still complicate reasoning. State may change between the point where a function begins waiting and the point where it resumes. Promises may reject,

operations may complete in an unexpected order, and cancellation may require explicit design. TypeScript can ensure that a function handles values of declared shapes; it cannot by itself ensure that events occur in the intended order.

Memory remains automatically managed by the JavaScript runtime, but asynchronous design affects object lifetime. A pending promise and its closures may retain request state until completion or until references can be collected. Launching unbounded concurrent operations can therefore exhaust connections or memory even though no allocation is manually managed. A robust asynchronous design needs bounded concurrency, prompt error handling, and limited retained state.

Compared with Python, asynchronous TypeScript resembles `asyncio` more closely than threading or multiprocessing. Both `asyncio` and JavaScript promises are especially effective for I/O-bound workloads and both require tasks to yield rather than block their event loop. Python additionally makes process-based CPU parallelism available through its standard library. JavaScript environments provide workers for parallel computation, but workers have separate execution contexts and require message passing or explicitly shared buffers. In neither language does adding `async` convert a CPU-bound algorithm into parallel work.

Asynchronous TypeScript is therefore strongest for applications that coordinate many typed I/O operations, such as web clients and network services. Its main risks appear at the boundaries between compile-time and runtime data and between one asynchronous stage and the next. Types improve the representation of those stages, while disciplined promise composition controls their ordering.

7 Comparative Analysis

Table 1 summarizes the main trade-offs. The comparison separates language mechanisms from style: functional Python changes how values are handled but retains Python’s memory manager and runtime type system.

Memory management shows that source-level style matters even when runtimes provide garbage collection. The parser’s mutable token list and cursor are compact and

Criterion	Imperative Python	Strict functional Python	Asynchronous TypeScript
State	Direct mutation and ordered statements	New values, recursion, and composition	State crosses promise and event boundaries
Type safety	Dynamic runtime types; optional hints	Same Python type system	Static checking; types erased at runtime
Memory	Automatic; in-place updates can limit copying	Automatic; immutability may increase allocation	Automatic; pending work may retain closures and state
Concurrency	Threads, processes, and coroutines	Purity reduces shared-state hazards but is not a scheduler	Event-loop concurrency with workers for parallelism
Primary strength	Clear direct control	Predictable transformations	Typed coordination of I/O-bound work
Primary limitation	Shared mutation increases coupling	Strict constraints may hurt clarity or efficiency	Ordering, rejection, and runtime data remain concerns

Table 1: Comparison of the three evaluated approaches.

efficient because they are local to a single operation. Once a node is constructed, its frozen data class prevents later mutation. This is a pragmatic boundary: mutation is used while building, and immutability is used for the durable representation. The functional portfolio applies immutability throughout and gains stronger guarantees about input preservation, but repeated slicing, concatenation, and dictionary unions allocate more intermediate values. Asynchronous TypeScript adds another lifetime concern because pending operations retain the state needed to resume. The correct question is therefore not simply whether memory is automatic, but how the program’s structure creates, shares, copies, and retains objects.

Type safety produces a similarly qualified result. Python annotations in both artifacts document useful contracts. The **Expression** union identifies every legal AST node, generic type variables describe reusable collection functions, and return annotations clarify whether an operation produces a list, dictionary, number, or callable. These declarations improve maintainability even though the runtime does not enforce them. TypeScript moves more of this checking into the normal compilation workflow, which is valuable when asynchronous values pass through several stages. However, erased types cannot validate a malformed API response. Both languages ultimately require runtime checks at trust boundaries; TypeScript provides stronger static checking between those boundaries.

Concurrency highlights the practical value of functional ideas. Shared mutable state is difficult because results can depend on timing and interleaving. Pure transformations of independent values reduce this risk in threads, processes, coroutines, or promise-based code. Yet purity is not concurrency. The functional portfolio executes sequentially, and recursion does not create parallel work. Conversely, asynchronous TypeScript provides concurrency but does not guarantee purity: callbacks and resumed functions can freely mutate shared module or object state. A strong concurrent design can combine the approaches by using asynchronous orchestration around small, mostly pure transformations.

The parser demonstrates why an exclusively functional or imperative classification can also be misleading. Its control flow is imperative, which makes token consumption easy to follow. Its output is immutable, which makes AST nodes safe to share and traverse. Its type hints communicate structural alternatives, and its exceptions clearly stop invalid parses. Rewriting every cursor movement as recursion over new token slices might conform more strictly to functional rules but would not necessarily improve the artifact. The current combination chooses techniques according to each subproblem.

The functional portfolio makes the opposite contribution. By refusing ordinary loops and mutation, it exposes costs that mixed-paradigm Python usually hides. `map`, `filter`, and `reduce` are excellent when they match the conceptual operation. Recursion is elegant for grammar trees and recursively defined mathematics. Persistent reconstruction is valuable when old and new values must coexist. None is automatically best for every collection traversal. The exercise succeeds precisely because its strictness creates evidence for evaluating the paradigm rather than merely asserting preferences about it.

The comparison therefore supports situational choices. An imperative Python loop is appropriate for a local, stateful computation where order is the clearest explanation. Functional Python techniques are appropriate for transformations, reusable pipelines, and code where limiting shared state improves testability. Asynchronous TypeScript is appropriate when a typed application must coordinate many I/O operations without blocking its main execution agent. CPU-intensive work calls for parallel facilities rather than merely asynchronous syntax. Untrusted data calls for runtime validation regardless of static annotations. These are complementary engineering decisions rather than competing identities.

8 Development Process and Verification

The [project repository](#) records the work as staged artifacts rather than a single final code dump. The parser directory contains its implementation, README, and focused `unittest` suite. The functional directory follows the same layout and adds an AST-based constraint check. Root documentation identifies the weighted deliverables and integrates them into the independent-study project. Git history provides versioned evidence that the proposal, activities, and evaluation documentation were developed in stages.

The complete test suite was executed on August 13, 2026, with Python 3.12.13. The parser/lexer suite passed all eleven tests in 0.002 seconds, and the functional suite passed all nine tests in 0.007 seconds. Together, the artifacts passed twenty of twenty test methods with no failures or errors. A representative parser execution using `2 + 3 * (x - 4)^2` also exited successfully and returned a JSON AST whose root was addition, whose right child was multiplication, and whose nested exponentiation applied to `(x - 4)` and 2. The exact commands and captured summaries are retained in `final/test-results.md`.

The suites can be reproduced with Python 3.10 or newer:

```
Set-Location Week1-2
python -m unittest -v
Set-Location ../week2-3
python -m unittest -v
```

This distinction is part of sound technical reporting. A readable test suite is evidence of intended verification, while this observed successful run is evidence about the tested revision and Python 3.12.13 environment. It does not prove correctness for every possible input, but it confirms that all specified behaviors and structural constraints passed in the recorded run.

9 Conclusion

The three approaches examined in this project manage different forms of complexity. Imperative Python makes operational state and execution order explicit. The expression parser shows why that directness is useful: a cursor advances through tokens, grammar functions

mirror nonterminals, and errors interrupt invalid paths. The implementation then adopts immutable AST nodes, demonstrating that an imperative algorithm can still expose a stable functional-style result.

Strict functional Python moves attention from state changes to value transformations. The portfolio shows that pure functions and unchanged inputs improve isolation, composition, and testing. It also shows the cost of applying the rules without exception in Python: recursion consumes stack frames, persistent updates allocate intermediate values, and compressed expressions can become harder to read than loops. The constraints are valuable when they make reasoning safer, but their value must exceed their runtime and readability costs.

Asynchronous TypeScript combines compile-time structural checking with JavaScript's event-loop model. It is particularly well suited to coordinating independent I/O operations, but its types disappear at runtime and its concurrency does not imply CPU parallelism. Runtime validation, error handling, bounded work, and explicit ordering remain necessary.

The central conclusion is therefore not that one paradigm wins. Language and paradigm choices change where programmers encounter complexity: mutable state, recursive reconstruction, static contracts, runtime boundaries, or asynchronous ordering. Effective design places each form of complexity where it is easiest to control. For this project, that means imperative parsing with immutable AST output, functional transformations where purity improves reasoning, and asynchronous typed orchestration when a system must wait on many external operations. A multi-paradigm solution is not a compromise between purer alternatives; it is often the clearest response to a problem whose parts have different risks.

References

- [1] Microsoft, "TypeScript for the New Programmer," *TypeScript Handbook*. <https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html> (accessed Aug. 13, 2026).
- [2] Python Software Foundation, "Memory Management," *Python 3.14 Documentation*. <https://docs.python.org/3/c-api/memory.html> (accessed Aug. 13, 2026).

- [3] Python Software Foundation, "gc - Garbage Collector interface," *Python 3.14 Documentation*. <https://docs.python.org/3/library/gc.html> (accessed Aug. 13, 2026).
- [4] Python Software Foundation, "typing - Support for type hints," *Python 3 Documentation*. <https://docs.python.org/3/library/typing.html> (accessed Aug. 13, 2026).
- [5] Python Software Foundation, "asyncio - Asynchronous I/O," *Python 3 Documentation*. <https://docs.python.org/3/library/asyncio.html> (accessed Aug. 13, 2026).
- [6] Python Software Foundation, "Global interpreter lock," *Python Glossary*. <https://docs.python.org/3/glossary.html#term-global-interpreter-lock> (accessed Aug. 13, 2026).
- [7] MDN Web Docs, "JavaScript execution model." https://developer.mozilla.org/en-US/docs/Web/JavaScript/Event_loop (accessed Aug. 13, 2026).
- [8] MDN Web Docs, "await." <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await> (accessed Aug. 13, 2026).
- [9] MDN Web Docs, "Using promises." https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises (accessed Aug. 13, 2026).